59. CAMPUS TRANSPORT ALLOCATION

1. AIM

To determine the required fleet of buses for each campus route based on passenger counts and bus seating capacity.

2. PROBLEM STATEMENT

College transport managers need to allocate buses across multiple campus routes. Write a Python program that accepts the number of routes, bus capacity, and passenger count per route, and calculates buses needed using integer division and modulus.

3. OBJECTIVES

- 1. Input total routes and bus capacity.
- 2. Compute buses = passengers // capacity.
- 3. Increment buses by 1 if remaining passengers exist (passengers % capacity != 0).
- 4. Display allocated buses per route.

4. ASSUMPTIONS & REQUIREMENTS

Assumptions:

- Bus capacity is a positive integer > 0.
- Passenger count is non-negative.

Requirements:

- Python 3.x Interpreter
- Integer division (`//`) and Modulus (`%`)

5. ALGORITHM

Step 1: Read routes and bus capacity.

Step 2: Loop i from 1 to routes:

a. Read passengers for route.

b. If capacity <= 0: print 'Invalid bus capacity'.

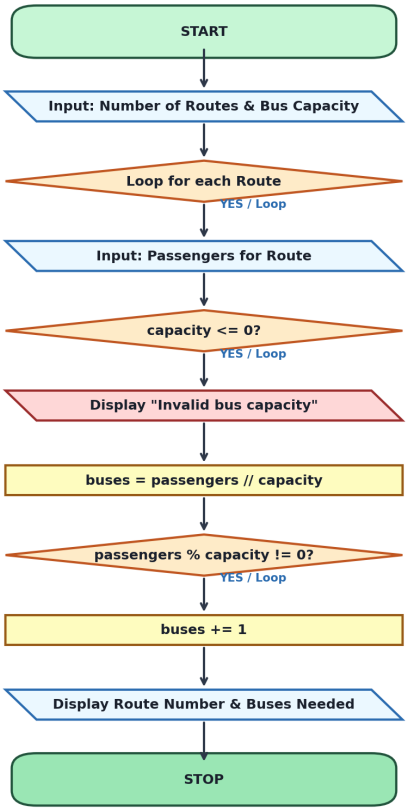
c. Else: buses = passengers // capacity.

d. If passengers % capacity != 0: buses += 1.

e. Print Route number and Buses needed.

Step 3: Stop.

6. PROGRAM FLOWCHART



7. PYTHON PROGRAM (transport.py)

```
routes = int(input("Enter number of routes: "))
capacity = int(input("Enter bus capacity: "))

for i in range(1, routes + 1):
    passengers = int(input("Enter passengers for route: "))

    if capacity <= 0:
        print("Invalid bus capacity")
    else:
        buses = passengers // capacity

        if passengers % capacity != 0:
            buses += 1

        print("Route:", i)
        print("Buses needed:", buses)
```

8. EXPLANATION OF THE CODE

- **passengers // capacity:** Determines fully occupied bus count.
- **passengers % capacity != 0:** Checks if an extra bus is required for remaining passengers.
- **buses += 1:** Adds an extra bus to accommodate overflow.

9. TRACING TABLE

Route	Passengers	Capacity	Full Buses (//)	Remainder (%)	Total Buses
Route 1	20	50	20 // 50 = 0	20 % 50 = 20 (!=0)	0 + 1 = 1

10. TEST CASES

Test Case	Input	Expected Result	Actual Result
TC 1 (Partial Load)	Routes: 2, Capacity: 50 Route 1 Passengers: 20	Route: 1 Buses needed: 1	Route: 1 Buses needed: 1
TC 2 (Exact Multiple)	Routes: 1, Capacity: 50 Passengers: 100	Route: 1 Buses needed: 2	Route: 1 Buses needed: 2
TC 3 (Invalid Capacity)	Routes: 1, Capacity: 0 Passengers: 30	Invalid bus capacity	Invalid bus capacity

11. EXPECTED OUTPUT

```
Enter number of routes: 2
Enter bus capacity: 50
Enter passengers for route: 20
Route: 1
Buses needed: 1
Enter passengers for route:
```

12. ACTUAL OUTPUT

```
Enter number of routes: 2
Enter bus capacity: 50
Enter passengers for route: 20
Route: 1
Buses needed: 1
Enter passengers for route:
```

13. PROGRAM EXECUTION SCREENSHOT

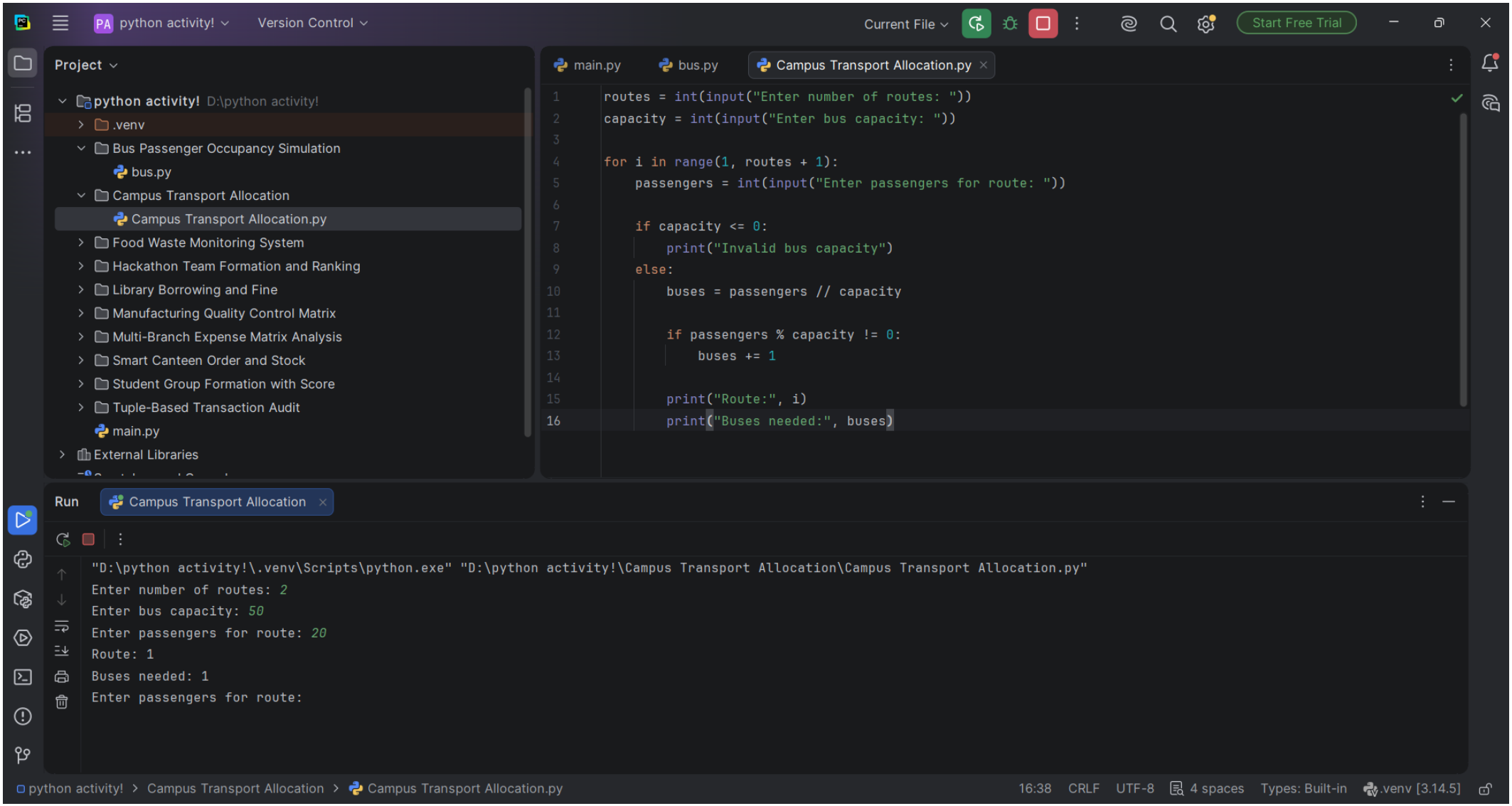
[Screenshot to Be Added]

14. RESULT / CONCLUSION

The Python program transport.py was executed successfully. Bus fleet allocation math performed as expected.

59 Campus Transport Allocation

Actual Input and Output – PyCharm Run Screenshot



Screenshot includes the program code and the console inputs/outputs.